Introduction to Data Science
R exercises in-class
Week 3

Dr. Gah-Yi Ban
Imperial College Business School

Tidyverse continued

Let us start by calling the `tidyverse` library.

NOTE: If you don't have the tidyverse package on your RStudio, please first install it by running `install.packages("tidyverse")` in the R Console.

```
library(tidyverse)
```

Before starting any practice and exercises, load the dataset "uk_gdp_and_weeklyWage.csv". Recall this gives us data on the average weekly pay (in £) and the total GDP (in millions of £) from 2000 to 2022.

```
uk_gdp_and_weeklyWage <- read.csv("./data/uk_gdp_and_weeklyWage.csv",
                                  stringsAsFactors = F)

head(uk_gdp_and_weeklyWage)
```

```
##   Year Weekly_pay   GDP_m
## 1 2000        296 1627447
## 2 2001        311 1662558
## 3 2002        323 1691998
## 4 2003        334 1744840
## 5 2004        346 1785756
## 6 2005        360 1833406
```

With pipe operators, you may use one or more of the following verbs (note there are others):

- `select()`: pick variables based on their names.
- `filter()`: pick cases based on their values.
- `mutate()`: add new variables that are functions of existing variables.
- `arrange()` allows you to sort a data frame by a particular column.
- `summarise()`: reduce multiple values down to a single summary.
- These all combine naturally with `group_by()` which allows you to perform any operation "by group".

Note If you use `group()`, then `ungroup()` is always used after performing all the calculations. If you forget to `ungroup()` data, future data management will likely produce errors. Always `ungroup()` swhen you've finished with your calculations.

mutate

Let us add GDP growth rates and categorize the growth rate into quartiles.

Note: there will be NA(s) in the GDP growth rates data

To do so, we need to make use of `cut()` and `quantile()` functions.

The `cut()` function in R is used to cut a range of values into bins and specify labels for each bin. This function uses the following syntax:

cut(x, breaks, include.lowest = TRUE or FALSE, labels = ...)

where:

- x: Name of vector
- breaks: Number of breaks to make or vector of break points
- labels: Labels for the resulting bins. `labels = NULL` produces interval bins and `labels = FALSE` produces integer category labels starting at 1. Otherwise, you can assign your own labels with `labels = your own vector of labels`.
- include.lowest: Boolean value to include lowest break value

To see how `cut()` works, study and run the code below that puts a vector of numbers from 0 to 100 into 5 bins.

```
vec <- c(0:100)

cut(vec, c(0,20,40,60,80,100), include.lowest = TRUE, labels = FALSE)
```

```
##   [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
##  [38] 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4
##  [75] 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

The `quantile()` function produces sample quantiles corresponding to the given probabilities. The smallest observation corresponds to a probability of 0 and the largest to a probability of 1.

The syntax is:

quantile(x, probs, na.rm = TRUE or FALSE, type = 7)

where:

- x: Numeric vector whose sample quantiles are wanted. NA and NaN values are not allowed in numeric vectors unless na.rm is set to TRUE.
- probs: Numeric vector of probabilities. E.g., if `probs = seq(0, 1, 0.25)`, then you are asking for the quartiles (0%, 25%, 50%, 75% and 100%).
- na.rm = TRUE or FALSE: if TRUE, any NA and NaN's are removed from x before the quantiles are computed.
- type: An integer between 1 and 9 selecting one of the nine quantile algorithms available are to be used. Default is to keep it at 7.

To see how `quantile()` works, study and run the code below that puts a vector of numbers from 0 to 100 into quartiles.

```r
vec <- c(0:100)

quantile(vec, probs = seq(0, 1, 0.25), type = 7)
```

```
##   0%  25%  50%  75% 100%
##    0   25   50   75  100
```

Now we are ready to use `mutate` to add GDP growth rates and categorize the growth rate into quartiles. Study and run the code chunk below.

```r
uk_gdp_and_weeklyWage <- uk_gdp_and_weeklyWage %>%
  mutate(GDP_growth_rate = (GDP_m - lag(GDP_m)) / lag(GDP_m) * 100,
         )

uk_gdp_and_weeklyWage <- uk_gdp_and_weeklyWage %>%
  mutate(GDP_growth_rate_Quartile = cut(GDP_growth_rate,
                                        quantile(GDP_growth_rate,
                                                 probs = seq(0, 1, 0.25), type = 7,
                                                 na.rm = T),
                                        include.lowest = T, labels = F)
          )

head(uk_gdp_and_weeklyWage)
```

```
##   Year Weekly_pay   GDP_m GDP_growth_rate GDP_growth_rate_Quartile
## 1 2000        296 1627447              NA                       NA
## 2 2001        311 1662558        2.157428                        2
## 3 2002        323 1691998        1.770765                        2
## 4 2003        334 1744840        3.123053                        4
## 5 2004        346 1785756        2.344971                        3
## 6 2005        360 1833406        2.668338                        4
```

arrange

Let's sort the data by the order of GDP growth rates using arrange.

```r
uk_gdp_and_weeklyWage %>% arrange(GDP_growth_rate) %>% head()
```

```
##   Year Weekly_pay   GDP_m GDP_growth_rate GDP_growth_rate_Quartile
## 1 2020        516 1991439     -11.0308585                        1
## 2 2009        412 1831550      -4.5104856                        1
## 3 2008        405 1918064      -0.1543444                        1
## 4 2011        427 1896087       1.0676109                        1
## 5 2012        434 1923551       1.4484567                        1
## 6 2019        505 2238348       1.6043087                        1
```

group_by

Let us now investigate the mean, standard deviation of GDPs, mean and standard deviation of weekly wage, and mean and standard deviation of GDP growth rates in each GDP growth rate quartile group by `summarise`.

Can you do this in the following code chunk?

Exercise 1

```
uk_gdp_and_weeklyWage %>%
  group_by(GDP_growth_rate_Quartile) %>%
  summarise(mean_GDP = mean(GDP_m), std_GDP = sd(GDP_m),
            mean_wage = mean(Weekly_pay), std_wage = sd(Weekly_pay),
            mean_GDPrate = mean(GDP_growth_rate),
            std_GDPrate = sd(GDP_growth_rate)) %>%
  ungroup()
```

```
## # A tibble: 5 x 7
##   GDP_growth_rate_Quartile mean_GDP std_GDP mean_wage std_wage mean_GDPrate
##                      <int>    <dbl>   <dbl>     <dbl>    <dbl>        <dbl>
## 1                        1 1966506. 142747.      450.     48.2        -1.93
## 2                        2 1877827. 219747.      387.     75.6         1.92
## 3                        3 2002378. 163360.      431.     51.7         2.36
## 4                        4 1982310. 185058.      440      97.7         3.88
## 5                       NA 1627447      NA       296      NA          NA
## # i 1 more variable: std_GDPrate <dbl>
```

After running the above code chunk, have you observed any issue(s)?

To address the issue(s), adjustment in the above code chunk can be done by different ways. For example, `filter` can be added to implement that in the Exercise 2.

Exercise 2

```
uk_gdp_and_weeklyWage %>%
  filter(!is.na(GDP_growth_rate_Quartile)) %>%
  group_by(GDP_growth_rate_Quartile) %>%
  summarise(mean_GDP = mean(GDP_m), std_GDP = sd(GDP_m),
            mean_wage = mean(Weekly_pay), std_wage = sd(Weekly_pay),
            mean_GDPrate = mean(GDP_growth_rate),
            std_GDPrate = sd(GDP_growth_rate)) %>%
  ungroup()
```

```
## # A tibble: 4 x 7
##   GDP_growth_rate_Quartile mean_GDP std_GDP mean_wage std_wage mean_GDPrate
##                      <int>    <dbl>   <dbl>     <dbl>    <dbl>        <dbl>
## 1                        1 1966506. 142747.      450.     48.2        -1.93
## 2                        2 1877827. 219747.      387.     75.6         1.92
## 3                        3 2002378. 163360.      431.     51.7         2.36
## 4                        4 1982310. 185058.      440      97.7         3.88
## # i 1 more variable: std_GDPrate <dbl>
```

Sometimes, viewing the first or last few rows of the data will be helpful to understand the data. Recall that we used `head` to display the first few rows of data.But we often need to sort the data before viewing.

Let us try viewing the first 10 rows of "uk_gdp_and_weeklyWage" after sorting the "GDP_growth_rate" with `arrange` in Exercise 3.

Exercise 3

```r
uk_gdp_and_weeklyWage %>% arrange(GDP_growth_rate) %>%
  head(10)
```

```
##    Year Weekly_pay   GDP_m GDP_growth_rate GDP_growth_rate_Quartile
## 1  2020        516 1991439     -11.0308585                        1
## 2  2009        412 1831550      -4.5104856                        1
## 3  2008        405 1918064      -0.1543444                        1
## 4  2011        427 1896087       1.0676109                        1
## 5  2012        434 1923551       1.4484567                        1
## 6  2019        505 2238348       1.6043087                        1
## 7  2018        488 2203005       1.7050210                        2
## 8  2002        323 1691998       1.7707653                        2
## 9  2013        438 1958557       1.8198634                        2
## 10 2001        311 1662558       2.1574282                        2
```

Now let us try viewing the last 10 rows of "uk_gdp_and_weeklyWage" after sorting the "GDP_growth_rate" with `arrange` and `tail` in Exercise 4.

Exercise 4

```r
uk_gdp_and_weeklyWage %>% arrange(GDP_growth_rate) %>%
  tail(10)
```

```
##    Year Weekly_pay   GDP_m GDP_growth_rate GDP_growth_rate_Quartile
## 14 2015        453 2069595        2.393103                        3
## 15 2010        420 1876058        2.430073                        3
## 16 2017        473 2166073        2.443570                        3
## 17 2007        390 1921029        2.563461                        4
## 18 2005        360 1833406        2.668338                        4
## 19 2003        334 1744840        3.123053                        4
## 20 2014        443 2021225        3.199703                        4
## 21 2022        571 2230625        4.101621                        4
## 22 2021        542 2142738        7.597471                        4
## 23 2000        296 1627447              NA                       NA
```

summarise

`summarise()` or `summarize()` function provides a summary of data, such as mean, standard deviation, median etc. The function is often used together with `group_by` if the data contain groups.

Let's start with a simply task by showing the mean of weekly wage, GDP and GDP growth rate.

```r
uk_gdp_and_weeklyWage %>%
    summarise(mean_wage = mean(Weekly_pay), mean_GDP = mean(GDP_m),
              mean_GDP_growth = mean(GDP_growth_rate))
```

```
##   mean_wage mean_GDP mean_GDP_growth
## 1  422.8261  1944407              NA
```

Have you found any issue after running the code above? Why? What's the solution?

You can handle NA values by adding an argument `na.rm=TRUE` or `na.rm=T` (for shorter version) into the function `mean`. Try these below.

Exercise 5

```r
uk_gdp_and_weeklyWage %>%
    summarise(mean_wage = mean(Weekly_pay), mean_GDP = mean(GDP_m),
              mean_GDP_growth = mean(GDP_growth_rate, na.rm=T))
```

```
##   mean_wage mean_GDP mean_GDP_growth
## 1  422.8261  1944407        1.503125
```

If we run `summary()` on the GDP growth rate column, we get 2.163% as the median rate. Let us extract this number from the string output using the `parse_number()` function, as below. To find out how to use `parse_number()`, you can search online or type `?parse_number` in the Console.

```r
GDP_growth_rate_summary <- uk_gdp_and_weeklyWage %>%
  summary(GDP_growth_rate, na.rm=T)

GDP_growth_rate_summary
```

```
##       Year        Weekly_pay        GDP_m         GDP_growth_rate
##  Min.   :2000   Min.   :296.0   Min.   :1627447   Min.   :-11.031
##  1st Qu.:2006   1st Qu.:367.0   1st Qu.:1832478   1st Qu.:  1.629
##  Median :2011   Median :427.0   Median :1921029   Median :  2.163
##  Mean   :2011   Mean   :422.8   Mean   :1944407   Mean   :  1.503
##  3rd Qu.:2016   3rd Qu.:468.5   3rd Qu.:2092000   3rd Qu.:  2.533
##  Max.   :2022   Max.   :571.0   Max.   :2238348   Max.   :  7.597
##                                                   NA's   :1
##  GDP_growth_rate_Quartile
##  Min.   :1.00
##  1st Qu.:1.25
##  Median :2.50
##  Mean   :2.50
##  3rd Qu.:3.75
##  Max.   :4.00
##  NA's   :1
```

```r
GDP_growth_rate_median <- parse_number(GDP_growth_rate_summary[3,4],
                                       locale = locale(grouping_mark = " "))

GDP_growth_rate_median
```

```
## [1] 2.163
```

Let's group the GDP growth rate data by the benchmark at 2.163% and check the mean and standard deviation of the GDP growth rate.

```r
uk_gdp_and_weeklyWage <- uk_gdp_and_weeklyWage %>%
  mutate(lower_median = GDP_growth_rate < GDP_growth_rate_median)

uk_gdp_and_weeklyWage %>% group_by(lower_median) %>%
  summarise(mean_GDP_growth = mean(GDP_growth_rate, na.rm=T),
            std_GDP_growth = sd(GDP_growth_rate, na.rm = T)) %>%
  ungroup()
```

```
## # A tibble: 3 x 3
##   lower_median mean_GDP_growth std_GDP_growth
##   <lgl>                  <dbl>          <dbl>
## 1 FALSE                  3.18            1.56
## 2 TRUE                  -0.178           4.08
## 3 NA                   NaN              NA
```

Any issue(s) found from the code chunk above?

Please find a solution to address the issue in the following code chunk.

Exercise 6

```r
uk_gdp_and_weeklyWage <- uk_gdp_and_weeklyWage %>%
  mutate(lower_median = GDP_growth_rate < GDP_growth_rate_median)

uk_gdp_and_weeklyWage %>%
  group_by(lower_median) %>%
  filter(!is.na(GDP_growth_rate)) %>%
  summarise(mean_GDP_growth = mean(GDP_growth_rate, na.rm=T),
            std_GDP_growth = sd(GDP_growth_rate, na.rm = T)) %>%
  ungroup()
```

```
## # A tibble: 2 x 3
##   lower_median mean_GDP_growth std_GDP_growth
##   <lgl>                  <dbl>          <dbl>
## 1 FALSE                  3.18            1.56
## 2 TRUE                  -0.178           4.08
```

The End